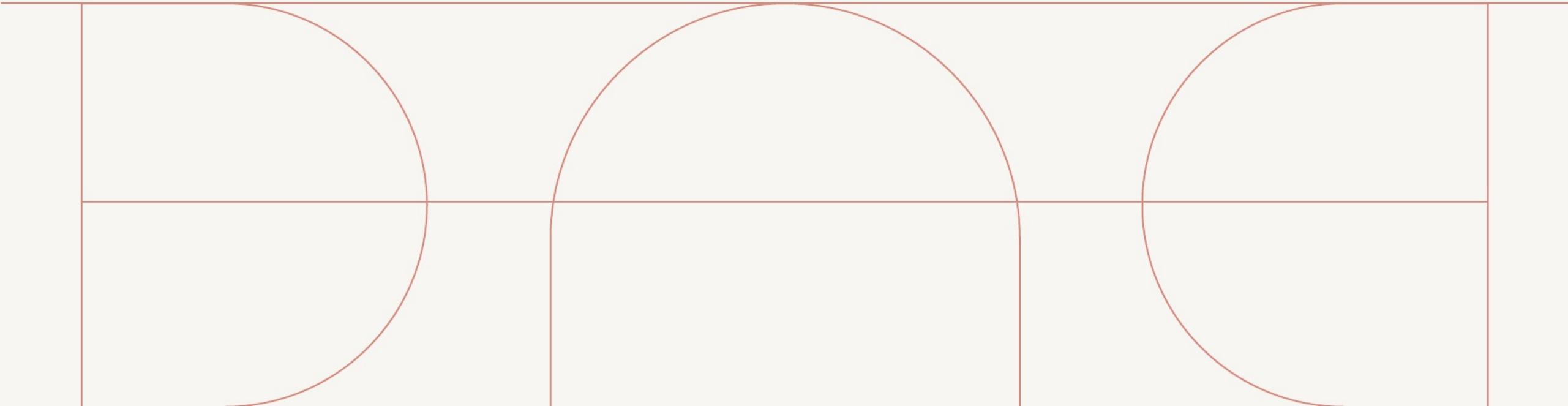
SPRING BATCH

TRABAJO POR LOTES



TRABJO POR LOTES

El procesamiento por lotes es una necesidad histórica en los Sistemas de Información. Algunos casos de uso comunes de los trabajos por lotes incluyen:

- Procesamiento de grandes conjuntos de datos
- Migración de bases de datos
- Generación de informes
- Operaciones ETL (Extracción, Transformación, Carga)

TRABJO POR LOTES

Para su realización se requieren de algunas características comunes como transacciones, lectura de entrada/salida, programación de tareas, procesamiento concurrente, secuenciación de tareas, etc.

Spring Batch es el módulo de Spring que da soporte a este tipo de procesos, facilitando su programación.

PRINCIPALES CLASES

La ontología de Spring para cubrir esta funcionalidad son típicamente

- Jobs: Trabajo o tarea principal, que puede implicar uno o vario pasos
- Steps: Paso o subtarea de un trabajo, asociado a un trabajo principal
- ItemReader: Proceso de lectura de un paso
- ItemProcesor: Procesamiento de cada item
- ItemWriter: Proceso de escritura de un paso

LECTURA/ESCRITURA

Spring proporciona diversas implementaciones por defecto para leer y escribir en distintas fuentes, como:

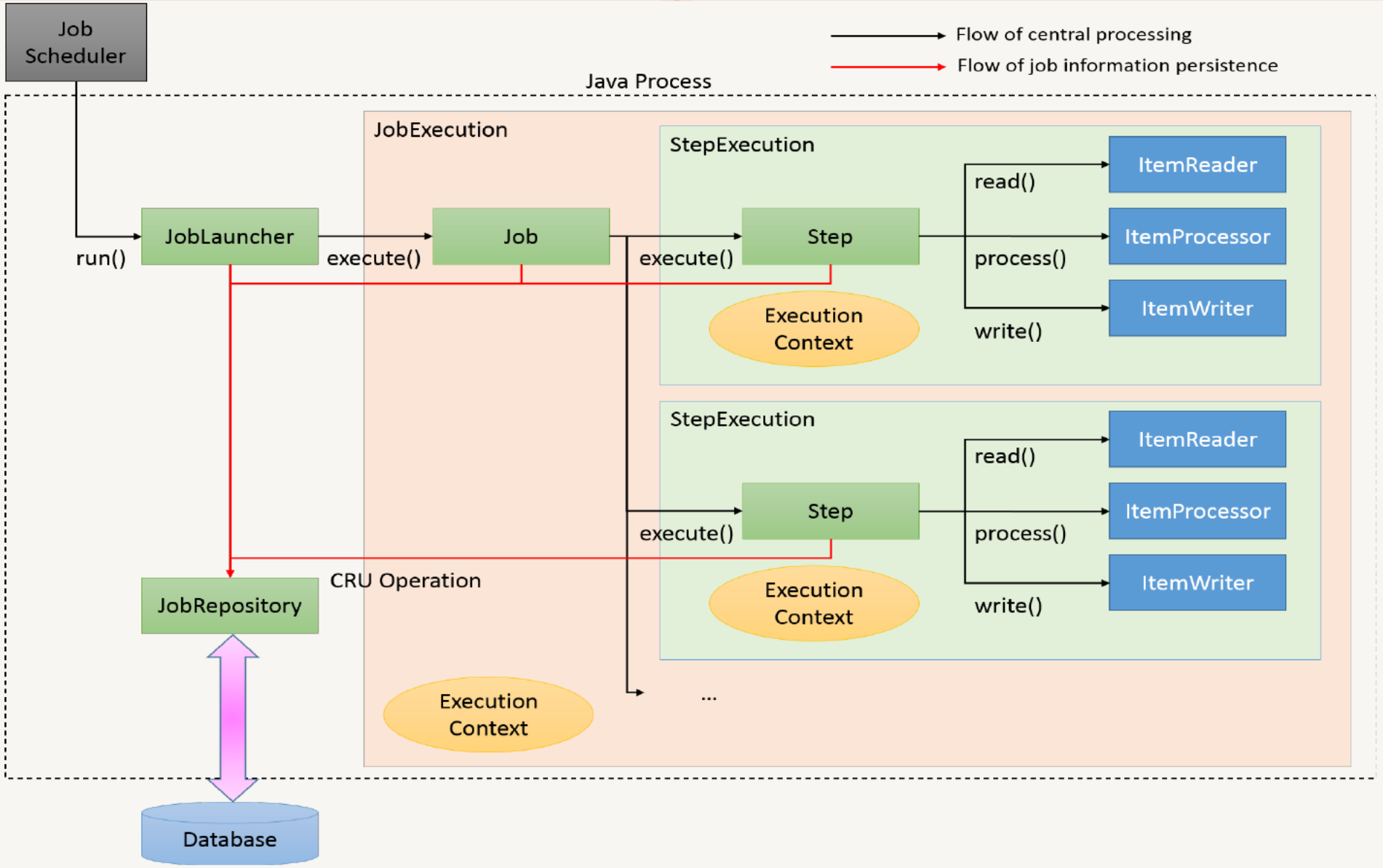
- Ficheros (CVS, JSON, XML)
- Bases de Datos (Relacionales y NOSQL)
- Colas de mensajes
- Apis

LECTURA/ESCRITURA

El procesamiento va orientado a una medida de lote, trozo o *chunk* , que es un subconjunto de los datos leidos, escritos o procesados, tratados bajo la misma transacción

En cada paso, los elementos son leídos y procesados uno por u hasta que se alcanza el límite de lote o *chunk*

Una vez relleno el chunk, se escribe conjuntamente como bloque


Job Scheduler
Flow of central processing
Flow of job information persistence
Java Process
JobExecution
StepExecution
ItemReader
read()
ItemProcessor
process()
ItemWriter
write()
JobLauncher
run()
execute()
Job
execute()
Step
Execution Context
StepExecution
ItemReader
read()
Step
execute()
ItemProcessor
process()
ItemWriter
write()
Execution Context
CRU Operation
JobRepository
Execution Context
...
Database

ASPECTOS AVANZADOS

PROGRAMACIÓN DE TAREAS

REPETICIÓN TAREAS FALLIDAS

LISTENERS

JOBS Y STEPS

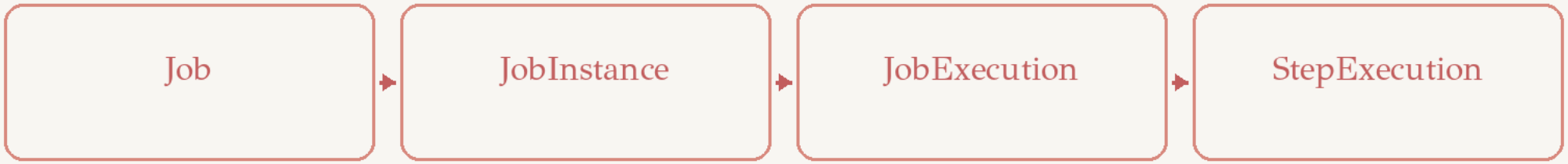
Un Job representa el proceso batch completo. Cada Job se divide en uno o varios Steps, responsables de realizar una parte concreta del trabajo.

- Los Steps se ejecutan siguiendo un orden y pueden finalizar con distintos estados.
- Los parámetros permiten lanzar un mismo Job con datos de entrada diferentes.

Idea clave: el Job define el flujo global; los Steps encapsulan sus unidades de trabajo.

EJECUCIÓN DE UN JOB

Spring Batch diferencia la definición del trabajo de sus instancias y de cada intento real de ejecución.



JobInstance identifica lógicamente una ejecución a partir del Job y sus parámetros. JobExecution representa cada intento real. StepExecution registra el resultado de cada Step.

TASKLETS

Un Tasklet implementa una tarea concreta que se ejecuta dentro de un Step. Es apropiado para operaciones unitarias que no necesitan procesamiento registro a registro.

- Preparar recursos o directorios.
- Validar la existencia de un fichero.
- Mover o archivar un fichero.
- Ejecutar una operación auxiliar.

El Tasklet devuelve un estado que indica si la tarea ha terminado o debe repetirse.

TASKLET O PROCESAMIENTO POR CHUNKS

Dos formas habituales de construir el trabajo de un Step.

TASKLET

- Una acción o tarea concreta.
- Se ejecuta como unidad.
- Adecuado para preparación, mantenimiento o control.
- Ejemplo: archivar un fichero.

CHUNK

- Procesamiento repetitivo de elementos.
- Lectura, transformación y escritura.
- Trabaja en bloques transaccionales.
- Ejemplo: importar 10.000 registros.

PROCESAMIENTO POR CHUNKS

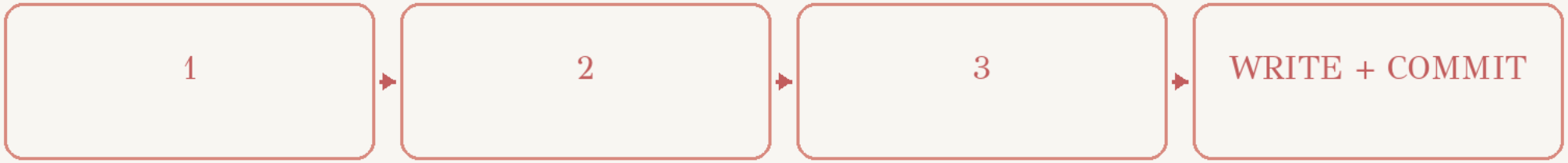
El ciclo Reader - Processor - Writer se repite hasta que no quedan elementos por procesar.



Reader obtiene cada elemento; Processor aplica validaciones, reglas o transformaciones; Writer realiza la salida o persistencia.

CHUNK Y TRANSACCIÓN

Ejemplo: chunk = 3. Se leen y procesan tres elementos; después se escriben juntos y se confirma la transacción.



Si se produce un error antes del commit, el bloque puede revertirse. El tamaño del chunk debe elegirse según volumen, memoria, E/S y comportamiento transaccional.

LECTURA DE DATOS

ItemReader abstrae la obtención de datos desde una fuente de entrada. Spring Batch dispone de implementaciones para ficheros, bases de datos y otras fuentes.

- CSV o texto delimitado.
- JDBC / SQL.
- XML / JSON.
- Otras fuentes mediante implementaciones personalizadas.

En ficheros delimitados, cada línea debe convertirse en un objeto de dominio.

TRANSFORMACIÓN DE DATOS

ItemProcessor recibe un elemento leído y devuelve el elemento que será escrito. Permite transformar formatos, normalizar datos, validar información y aplicar reglas de negocio.

- Puede convertir tipos y formatos.
- Puede normalizar o enriquecer datos.
- Puede descartar elementos que no deban continuar.

Ejemplo: texto con fecha e importe -> validación y conversión -> objeto listo para persistir.

ESCRITURA DE DATOS

ItemWriter recibe los elementos procesados agrupados en el chunk y realiza la salida o persistencia.

FICHEROS

- Generación de CSV, texto, XML u otros formatos.
- La escritura debe ser coherente con reinicios y repeticiones.

BASE DE DATOS

- Inserciones o actualizaciones por lotes mediante JDBC.
- La escritura suele participar en la misma transacción del chunk.

LECTURA, TRANSFORMACIÓN Y ESCRITURA

Flujo completo de un Step orientado a chunks.



Ejemplo formativo: leer clientes, validar datos, normalizar contenido, insertar registros correctos y registrar rechazados.

GESTIÓN DE ERRORES

Un proceso batch debe distinguir entre errores recuperables y errores que obligan a detener la ejecución.

SKIP

- Dato incorrecto: se descarta el elemento y el proceso continúa.
- Un límite evita continuar indefinidamente ante demasiados errores.

RETRY

- Fallo temporal: se vuelve a intentar la operación.
- Un límite define cuántos reintentos se permiten.

REINICIO DE EJECUCIONES

Spring Batch persiste metadatos para conocer qué ocurrió en cada Job y Step. El ExecutionContext puede conservar información necesaria para continuar un proceso.

- Los Steps ya completados pueden evitar repetirse.
- El proceso puede continuar desde el punto adecuado.
- Readers, Writers y operaciones auxiliares deben diseñarse de forma consistente.

Objetivo: recuperar una ejecución interrumpida evitando repetir innecesariamente todo el procesamiento.

LISTENERS

Los listeners permiten reaccionar a eventos del ciclo de vida del Job, del Step o de los elementos procesados.

- Antes o después de un Job o Step.
- Eventos de lectura, procesamiento y escritura.
- Errores, logging, auditoría, estadísticas y notificaciones.

Ejemplo: al finalizar un Job, registrar duración, estado y contadores de registros.

TRAZABILIDAD

La trazabilidad permite saber qué se ejecutó, cuándo se ejecutó y con qué resultado.

- Identificar ejecuciones mediante nombre del Job y parámetros.
- Registrar fechas de inicio y fin, estado final y errores.
- Controlar contadores de lectura, proceso, escritura, skip y rollback.
- Combinar metadatos de Spring Batch con logs de aplicación.

Preguntas clave: ¿qué Job se lanzó?, ¿con qué parámetros?, ¿qué Step falló?, ¿cuántos registros se procesaron?

TESTING

Los procesos batch deben probarse tanto a nivel de componentes como de ejecución completa.

- Probar Reader, Processor y Writer de forma aislada.
- Ejecutar Steps y Jobs con entradas controladas.
- Comprobar estados, resultados persistidos, contadores y errores.

Enfoque: preparar datos -> ejecutar -> comprobar estado y salida -> limpiar el entorno de prueba.

¿QUÉ DEBEMOS PROBAR?

- Que el Job termina con el estado esperado.
- Que los Steps se ejecutan en el orden previsto.
- Que los registros válidos se transforman y escriben correctamente.
- Que los registros erróneos siguen la política definida.
- Que un proceso fallido puede reiniciarse de forma coherente.
- Que los contadores y datos finales coinciden con el escenario de prueba.

Un proceso batch no está completo hasta que su comportamiento normal, sus errores y su recuperación son verificables.